8.1.3 파라체인 템플릿으로 시작하기

개요

이 튜토리얼에서는 Cumulus 파라체인 템플릿을 사용하여 첫 번째 파라체인을 개발하고 로컬 테스트넷에서 실행하는 과정을 학습합니다. 파라체인의 기본 개념부터 실제 배포까지 단계별로 진행하며, 릴레이 체인과의 연결 과정도 함께 다룹니다.

학습 목표

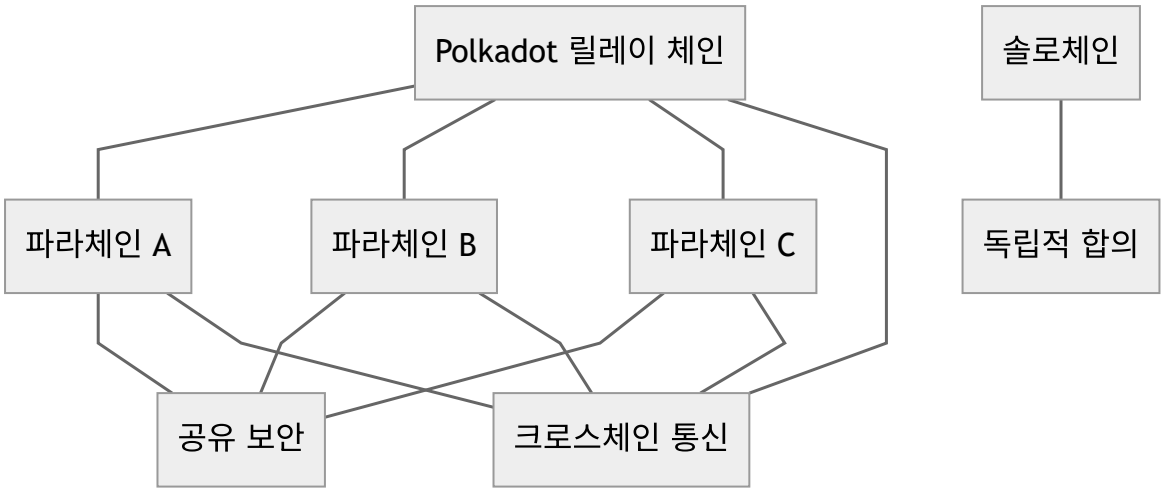
- 파라체인과 릴레이 체인의 관계 이해
- Cumulus 파라체인 템플릿 구조 파악
- 로컬 테스트넷 환경 구성
- 파라체인 등록 및 연결 과정 실습
- 기본적인 XCM 메시지 전송 테스트

사전 요구사항

- Substrate 노드 실행 경험
- 팔렛 개발 기본 지식
- Docker (선택사항, 간편한 환경 구성용)
- 최소 16GB RAM, 100GB 디스크 공간

1단계: 파라체인 개념 이해

1.1 파라체인 vs 솔로체인



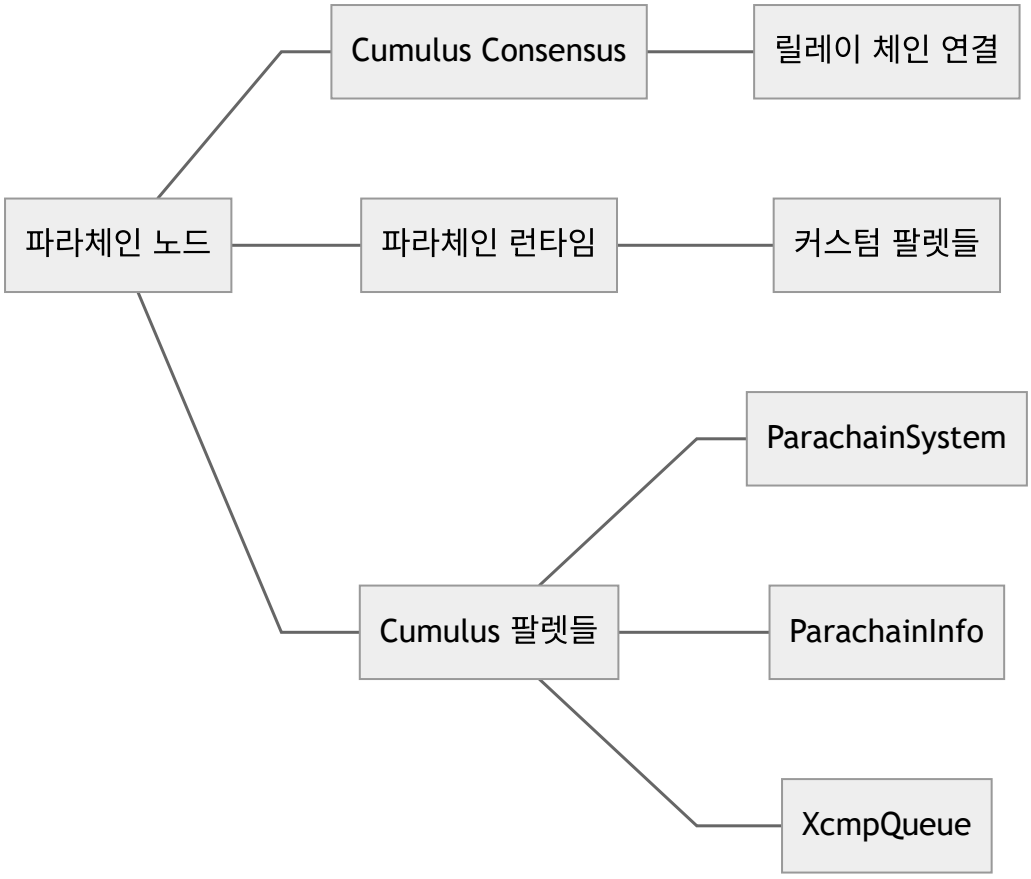
파라체인의 장점:

- 릴레이 체인의 공유 보안 활용
- 크로스체인 상호운용성 (XCM)
- 합의 메커니즘 구현 불필요
- 업그레이드 용이성

파라체인의 제약:

- 슬롯 경매를 통한 제한된 슬롯
- 릴레이 체인 의존성
- 추가적인 복잡성

1.2 Cumulus 아키텍처



2단계: 개발 환경 준비

2.1 Polkadot 릴레이 체인 바이너리 준비

```
# 작업 디렉토리 생성
mkdir ~/parachain-projects
cd ~/parachain-projects

# Polkadot 바이너리 다운로드 (또는 빌드)
# 방법 1: 사전 빌드된 바이너리 다운로드
wget https://github.com/paritytech/polkadot/releases/download/v1.0.0/polkadot
chmod +x polkadot
```

```
# 방법 2: 소스에서 빌드 (시간이 오래 걸림)
git clone https://github.com/paritytech/polkadot.git
cd polkadot
cargo build --release
cp target/release/polkadot ../polkadot
cd ..
```

2.2 파라체인 템플릿 다운로드

```
# 파라체인 템플릿 클론
git clone https://github.com/substrate-developer-hub/substrate-parachain-templ
cd substrate-parachain-template

# 최신 버전으로 체크아웃
git checkout polkadot-v1.0.0
```

2.3 프로젝트 구조 확인

```
# 프로젝트 구조 확인
tree -L 2
```

예상 출력:

```
.
├── Cargo.toml
├── README.md
├── node/ # 파라체인 노드 구현
├── pallets/ # 커스텀 팔렛들
├── runtime/ # 파라체인 런타임
└── zombienet/ # 테스트넷 설정 파일들
```

주요 차이점 (솔로체인 vs 파라체인):

- `node/src/service.rs` : Cumulus 서비스 통합
- `runtime/src/lib.rs` : Cumulus 팔렛들 포함
- `node/src/command.rs` : 파라체인 특화 CLI 명령어

3단계: 파라체인 빌드

3.1 의존성 확인

```
# 루트 Cargo.toml 확인
cat Cargo.toml

# 런타임 의존성 확인
cat runtime/Cargo.toml | grep cumulus
```

주요 Cumulus 의존성들:

```
cumulus-pallet-aura-ext = { version = "0.1.0" }

cumulus-pallet-parachain-system = { version = "0.1.0" }

cumulus-pallet-xcm = { version = "0.1.0" }

cumulus-pallet-xcmp-queue = { version = "0.1.0" }

cumulus-primitives-core = { version = "0.1.0" }
```

3.2 파라체인 노드 빌드

```
# 릴리즈 모드로 빌드

cargo build --release


# 빌드 시간: 30-60분 (첫 번째 빌드)
```

3.3 빌드 결과 확인

```
# 빌드된 바이너리 확인

ls -la target/release/parachain-template-node


# 실행 권한 확인

chmod +x target/release/parachain-template-node


# 버전 정보 확인
```

```
./target/release/parachain-template-node  --version
```

4단계: 체인 스펙 생성

4.1 릴레이 체인 스펙 생성

```
# 로컬 테스트넷용 릴레이 체인 스펙 생성
./polkadot  build-spec  --chain  rococo-local  --disable-default-bootnode > roco

# Raw 스펙 생성
./polkadot  build-spec  --chain  rococo-local.json  --raw  --disable-default-boo
```

4.2 파라체인 스펙 생성

```
# 파라체인 스펙 생성
./target/release/parachain-template-node  build-spec  --disable-default-bootnode

# Raw 스펙 생성
./target/release/parachain-template-node  build-spec  --chain  para-2000-plain.j
```

4.3 제네시스 상태 및 WASM 추출

```
# 파라체인 제네시스 상태 추출
```

```
./target/release/parachain-template-node export-genesis-state --chain para-20

# 파라체인 WASM 런타임 추출
./target/release/parachain-template-node export-genesis-wasm --chain para-200
```

5단계: 로컬 테스트넷 실행

5.1 릴레이 체인 검증자 노드 시작

터미널 1 - Alice 검증자:

```
./polkadot \
--alice \
--validator \
--base-path /tmp/relay/alice \
--chain rococo-local-raw.json \
--port 30333 \
--rpc-port 9933 \
--ws-port 9944 \
--unsafe-rpc-external \
--unsafe-ws-external \
--rpc-cors all
```

터미널 2 - Bob 검증자:

```
./polkadot \
--bob \
--validator \
--base-path /tmp/relay/bob \
--chain  rococo-local-raw.json \
--port 30334 \
--rpc-port  9934 \
--ws-port 9945 \
--bootnodes  /ip4/127.0.0.1/tcp/30333/p2p/12D3KooWEyoppNCUx8Yx66oV9fJnriXwCcXwDD
```

5.2 릴레이 체인 연결 확인

```
# Alice 노드 로그에서 다음과 같은 메시지 확인:
# "💤 Idle (1 peers), best: #0 (0x...)"
# "🙌 Starting consensus session on top of parent 0x..."
# "🎁 Prepared block for proposing at 1"
```

5.3 파라체인 콜레이터 노드 시작

터미널 3 - 파라체인 콜레이터:

```
./target/release/parachain-template-node \
--alice \
```

```
--collator \
--force-authoring \
--chain  para-2000-raw.json \
--base-path /tmp/parachain/alice \
--port  40333 \
--rpc-port 8844 \
--ws-port  8845 \
--unsafe-rpc-external \
--unsafe-ws-external \
--rpc-cors all \
-- \
--execution wasm \
--chain  rococo-local-raw.json \
--port 30343 \
--rpc-port  9977 \
--ws-port 9988
```

명령어 설명:

- `--collator` : 콜레이터 모드로 실행
- `--force-authoring` : 강제 블록 생성
- `--` : 이후는 내장된 릴레이 체인 노드 설정
- 파라체인과 릴레이 체인 각각의 포트 설정

6단계: 파라체인 등록

6.1 Polkadot.js Apps 연결

1. https://polkadot.js.org/apps/ 접속
2. 좌측 상단에서 “Development” → “Local Node” 선택
3. WebSocket URL을 `ws://127.0.0.1:9944` 로 설정
4. “Switch” 버튼 클릭

6.2 파라체인 등록 트랜잭션 생성

Developer → Sudo 메뉴에서:

1. `parasSudoWrapper` → `sudoScheduleParaInitialize` 선택
2. 파라미터 설정:

- `id: 2000`
- `genesisHead: para-2000-genesis-state` 파일 내용 붙여넣기
- `validationCode: para-2000-wasm` 파일 내용 붙여넣기
- `paraKind: Yes` (파라체인)

```
# 파일 내용을 16진수로 변환하여 복사
xxd -p -c 256 para-2000-genesis-state
xxd -p -c 256 para-2000-wasm
```

3. “Submit Sudo” 버튼 클릭

6.3 등록 확인

Network → Parachains 메뉴에서:

- “Parachains” 탭에서 ID 2000 파라체인 확인

- 상태가 “Onboarding” → "Parachain"으로 변경되는지 확인

7단계: 파라체인 기능 테스트

7.1 파라체인 블록 생성 확인

```
# 파라체인 콜레이터 로그에서 확인:
# "🙌 Starting consensus session on top of parent 0x..."
# "🎁 Prepared block for proposing at 1"
# "✨ Imported #1 (0x...)"
```

7.2 파라체인 상태 조회

1. Polkadot.js Apps에서 파라체인에 연결:

- WebSocket URL: `ws://127.0.0.1:8845`

2. “Network” → "Explorer"에서 블록 생성 확인
3. “Developer” → "Chain state"에서 상태 조회

7.3 기본 트랜잭션 테스트

Accounts 메뉴에서:

1. Alice 계정에서 Bob 계정으로 토큰 전송
2. “Developer” → "Extrinsics"에서 커스텀 팔렛 함수 호출
3. 트랜잭션이 정상적으로 처리되는지 확인

8단계: XCM 메시지 테스트

8.1 XCM 설정 확인

```
// runtime/src/lib.rs에서 XCM 설정 확인
impl cumulus_pallet_xcm::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type XcmExecutor = XcmExecutor<XcmConfig>;
}

impl cumulus_pallet_xcmp_queue::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type XcmExecutor = XcmExecutor<XcmConfig>;
type ChannelInfo = ParachainSystem;
type VersionWrapper = ();
type ExecuteOverweightOrigin = EnsureRoot<AccountId>;
type ControllerOrigin = EnsureRoot<AccountId>;
type ControllerOriginConverter = XcmOriginToTransactDispatchOrigin;
type WeightInfo = ();
type PriceForSiblingDelivery = NoPriceForSiblingDelivery<ParaId>;
}
```

8.2 간단한 XCM 메시지 전송

Developer → Extrinsics에서:

1. polkadotXcm → send 선택

2. 파라미터 설정:


```
{
"dest": {
"V3": {
"parents": 1,
"interior": "Here"
}
},
"message": {
"V3": [
{
"ReportError": {
"queryId": "0",
"dest": {
"parents": 0,
"interior": "Here"
},
"maxResponseWeight": {
"refTime": "1000000000",
"proofSize": "0"
}
}
```

```
}
]
}
}
```

3. 트랜잭션 제출 및 결과 확인

8.3 XCM 메시지 모니터링

```
# 릴레이 체인과 파라체인 로그에서 XCM 관련 메시지 확인
# 파라체인 로그:
# "XCM message sent: ..."
# 릴레이 체인 로그:
# "XCM message received from parachain 2000"
```

9단계: Zombienet을 사용한 자동화

9.1 Zombienet 설치

```
# Zombienet 바이너리 다운로드
wget https://github.com/paritytech/zombienet/releases/download/v1.3.82/zombiene
chmod +x zombienet-linux-x64
mv zombienet-linux-x64 zombienet
```

9.2 네트워크 설정 파일 생성

```
# zombienet/small-network.toml

[settings]

timeout = 1000


[relaychain]

default_image = "parity/polkadot:latest"

default_command = "./polkadot"

default_args = [ "-lparachain=debug" ]

chain = "rococo-local"


[[relaychain.nodes]]

name = "alice"

validator = true


[[relaychain.nodes]]

name = "bob"

validator = true


[[parachains]]

id = 2000

chain = "local"
```

```
[parachains.collator]

name = "collator01"

image = "parachain-template:latest"

command = "./parachain-template-node"
```

9.3 Docker 이미지 빌드

```
# Dockerfile

FROM ubuntu:20.04


RUN apt-get update && apt-get install -y \

ca-certificates \

&& rm -rf /var/lib/apt/lists/*


COPY target/release/parachain-template-node /usr/local/bin/

COPY para-2000-raw.json /etc/


EXPOSE 30333 9933 9944 8844 8845


ENTRYPOINT ["/usr/local/bin/parachain-template-node"]


# Docker 이미지 빌드
```

```
docker build -t parachain-template:latest .
```

9.4 Zombienet으로 네트워크 시작

```
# 네트워크 시작
./zombienet spawn zombienet/small-network.toml

# 네트워크 상태 확인
./zombienet test zombienet/small-network.toml
```

10단계: 문제 해결 및 디버깅

10.1 일반적인 문제들

문제 1: 파라체인이 릴레이 체인에 연결되지 않음

```
# 해결 방법:
# 1. 릴레이 체인 스펙과 파라체인 설정 일치 확인
# 2. 포트 충돌 확인
# 3. 부트노드 주소 정확성 확인

# 릴레이 체인 피어 확인
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "system_peers", "params":[]}' \
```

```
http://localhost:9933/
```

문제 2: 파라체인 등록 실패

```
# 제네시스 상태와 WASM 파일 재생성
./target/release/parachain-template-node export-genesis-state --chain para-20
./target/release/parachain-template-node export-genesis-wasm --chain para-200

# 파일 크기 확인
ls -la para-2000-*
```

문제 3: 블록 생성 중단

```
# 콜레이터 로그 확인
tail -f /tmp/parachain/alice/chains/local_testnet/db/full.log

# 릴레이 체인 연결 상태 확인
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "system_health", "params":[]}' \
http://localhost:8844/
```

10.2 성능 모니터링

```
# 파라체인 메트릭 확인
```

```
curl  http://localhost:8844/metrics | grep  substrate_block_height


# 릴레이 체인 메트릭 확인
curl  http://localhost:9933/metrics | grep  substrate_block_height
```

10.3 로그 레벨 조정

```
# 상세한 로그로 파라체인 실행
./target/release/parachain-template-node \
--alice \
--collator \
--force-authoring \
--chain  para-2000-raw.json \
--base-path /tmp/parachain/alice \
-lruntime=debug \
-lcumulus-collator=debug \
-lxcm=debug \
-- \
--chain  rococo-local-raw.json
```

11단계: 다음 단계 및 확장

11.1 커스텀 팔렛 추가

```
// runtime/src/lib.rs에 커스텀 팔렛 추가

impl pallet_my_custom::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

// 추가 설정...

}



construct_runtime!(

pub  struct Runtime {

// ... 기존 팔렛들

MyCustom: pallet_my_custom,

}

);
```

11.2 XCM 고급 기능

- 크로스체인 자산 전송
- 원격 실행 (Remote Execution)
- 다중 홉 라우팅

11.3 실제 테스트넷 배포

- Rococo 테스트넷 연결
- 파라체인 슬롯 경매 참여
- 크라우드론 캠페인 설정

요약

이 튜토리얼에서는 다음을 학습했습니다:

1. **파라체인 개념**: 릴레이 체인과의 관계 및 Cumulus 아키텍처
2. **환경 구성**: 릴레이 체인과 파라체인 바이너리 준비
3. **로컬 테스트넷**: 검증자 노드와 콜레이터 노드 실행
4. **파라체인 등록**: Sudo를 통한 파라체인 등록 과정
5. **기능 테스트**: 블록 생성, 트랜잭션 처리, XCM 메시지 전송
6. **자동화 도구**: Zombienet을 사용한 네트워크 관리

다음 고급 튜토리얼에서는 DeFi 팔렛 구현, 크로스체인 자산 전송, 거버넌스 시스템 커스터마이징 등을 다루겠습니다.

참고 자료

- Cumulus 공식 문서
- Polkadot 파라체인 개발 가이드
- XCM 포맷 명세
- Zombienet 사용법
- Rococo 테스트넷 가이드